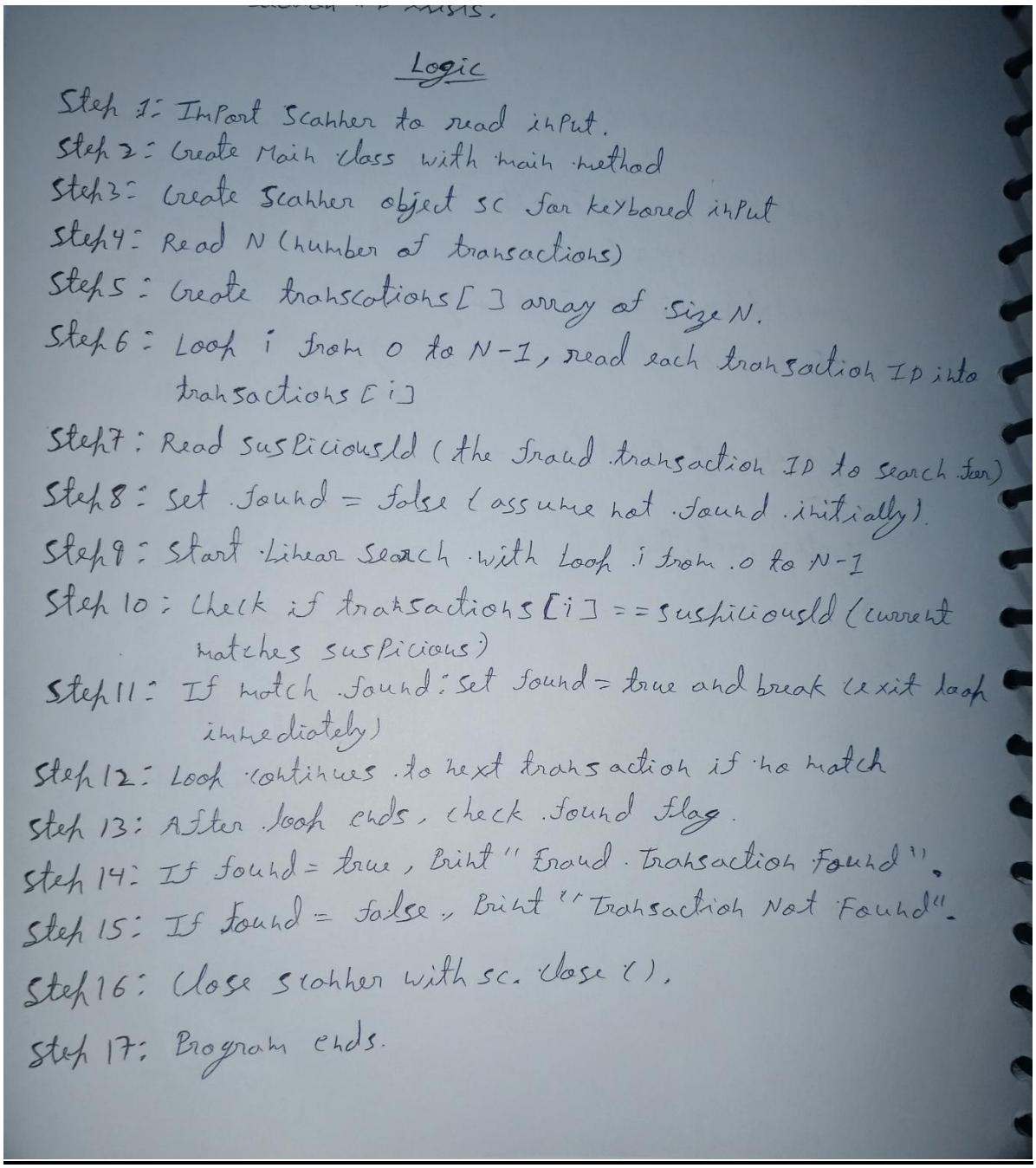
Week 2 Module:

1.Fraud transaction Detection (Linear Search)

Problem Statement

A bank store transaction IDs in the order they occur. Since the transaction are not sorted, the security team needs to check if a suspicious transaction ID exist.

Logic

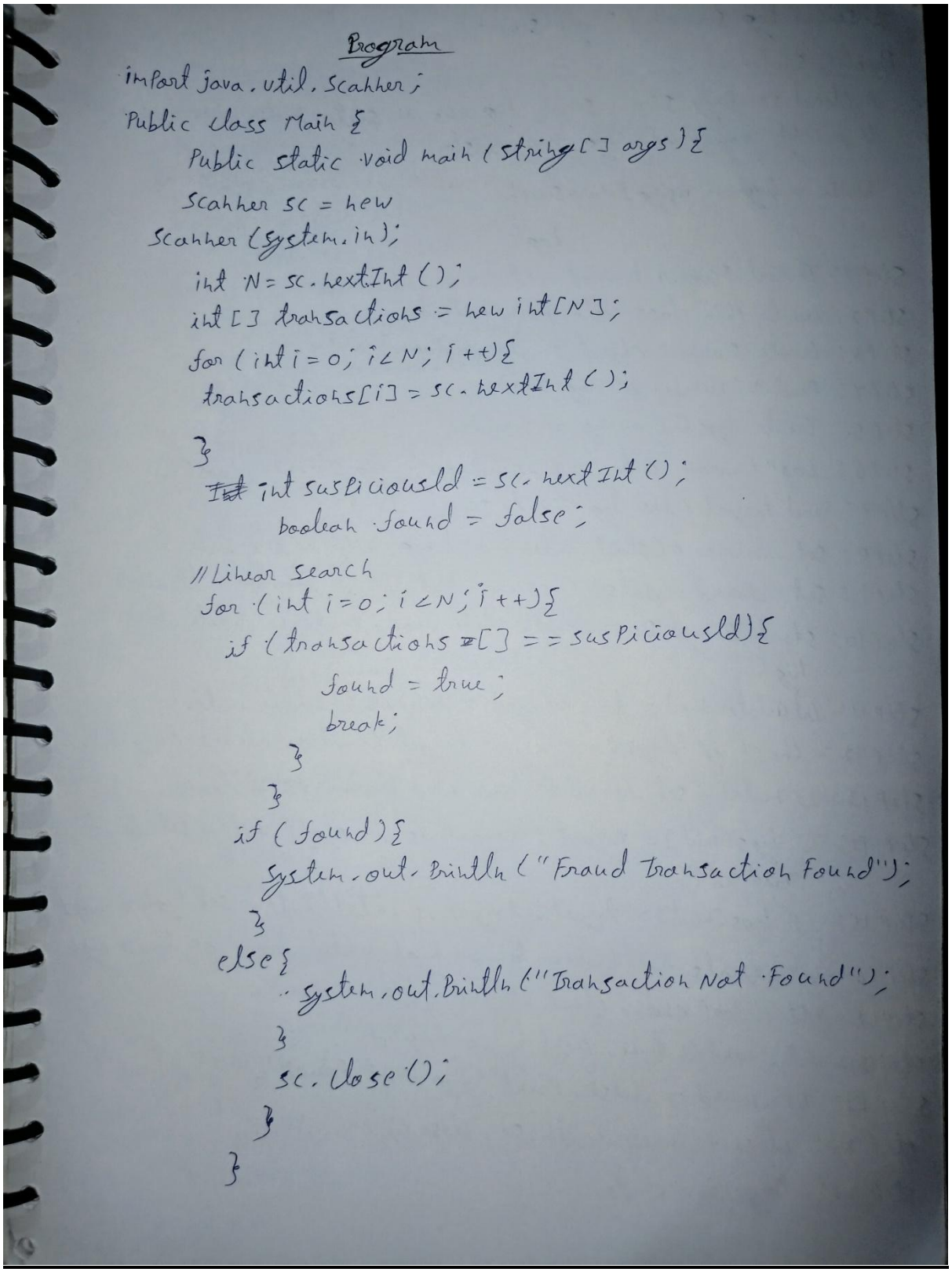


Logic

Step 1: Import Scanner to read input.
Step 2: Create Main class with main method
Step 3: Create Scanner object sc for keyboard input
Step 4: Read N (number of transactions)
Step 5: Create transcations[ ] array of size N.
Step 6: Loop i from 0 to N-1, read each transaction ID into transactions[i]
Step 7: Read suspiciousId (the fraud transaction ID to search for)
Step 8: Set found = false (assume not found initially).
Step 9: Start Linear search with loop i from 0 to N-1
Step 10: Check if transactions[i] == suspiciousId (current matches suspicious)
Step 11: If match found: set found = true and break (exit loop immediately)
Step 12: Loop continues to next transaction if no match
Step 13: After loop ends, check found flag.
Step 14: If found = true, Print "Fraud Transaction Found".
Step 15: If found = false, Print "Transaction Not Found".
Step 16: Close scanner with sc.close().
Step 17: Program ends.

Week 2 Module:

Program



Program

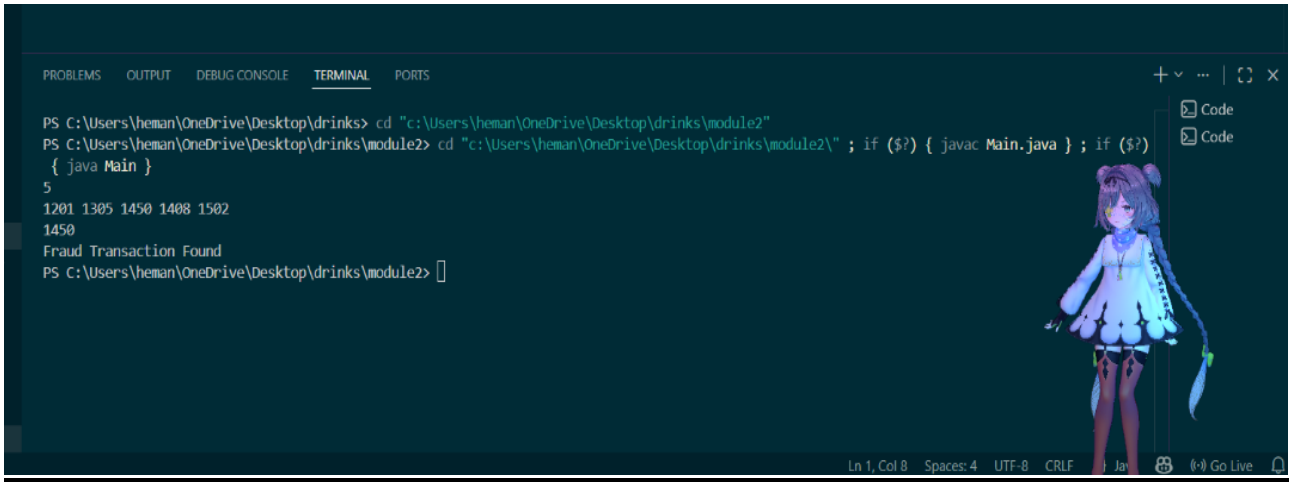
```
import java.util.Scanner;
Public class Main {
    Public static void main (String[] args){
    Scanner sc = new
  Scanner (System.in);
    int N= sc.nextInt();
    int[] transactions = new int[N];
    for (int i=0; i<N; i++){
     transactions[i] = sc.nextInt();

    }
     int suspiciousId = sc.nextInt();
         boolean found = false;

    //Linear search
    for (int i=0; i<N; i++){
      if (transactions[] == suspiciousId){
              found = true;
              break;
          }
       }
      if (found){
        System.out.println("Fraud transaction found");
      }
     else{
         System.out.println("Transaction Not Found");
       }
       sc.close();
     }
   }
```

Week 2 Module:

Output



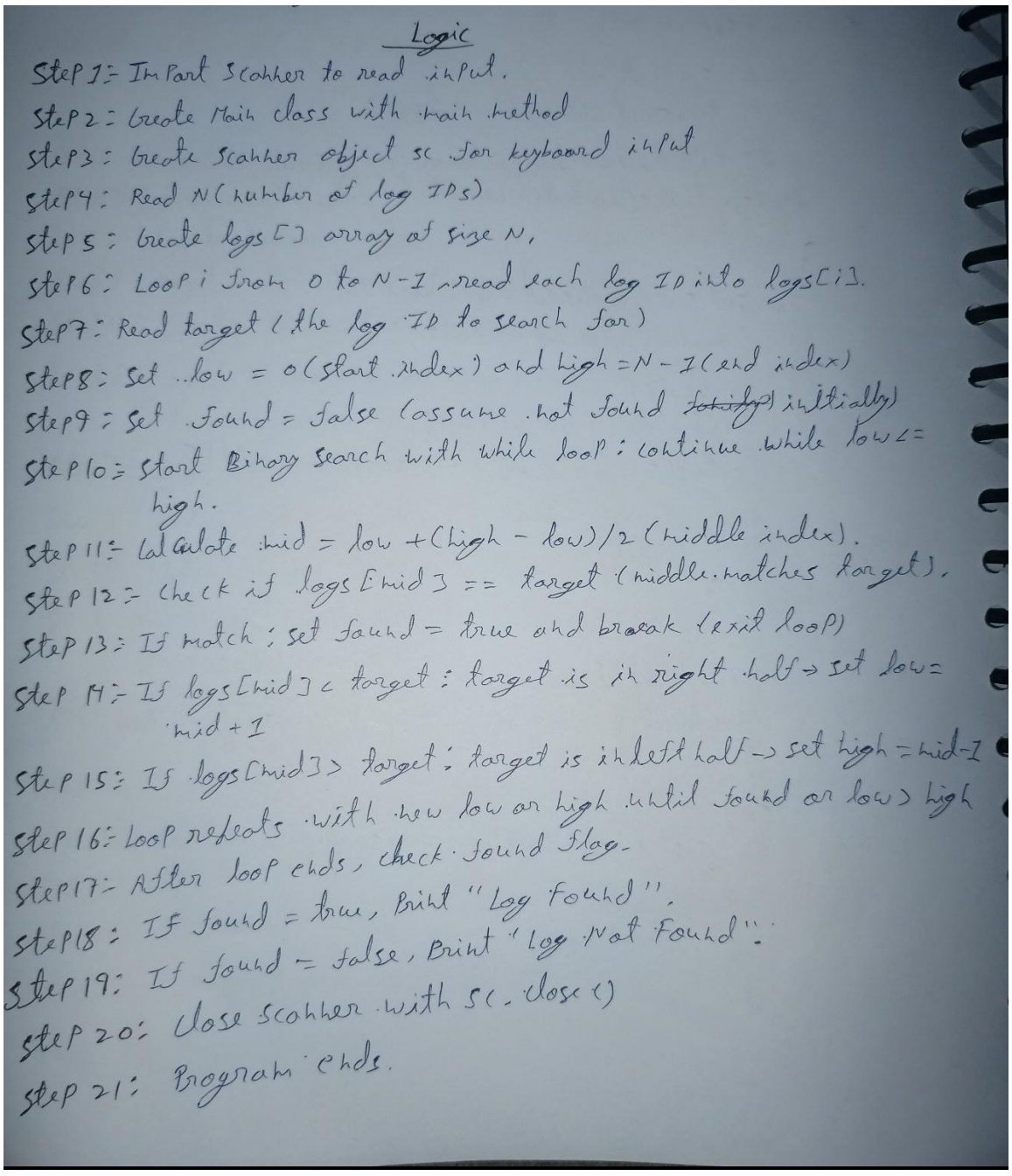
Week 2 Module:

2.Server Log Search (Binary Search)

Problem Statement

A cloud company stores log IDs in sorted order. Engineers need to quickly verify if a log IDs exists.

Logic

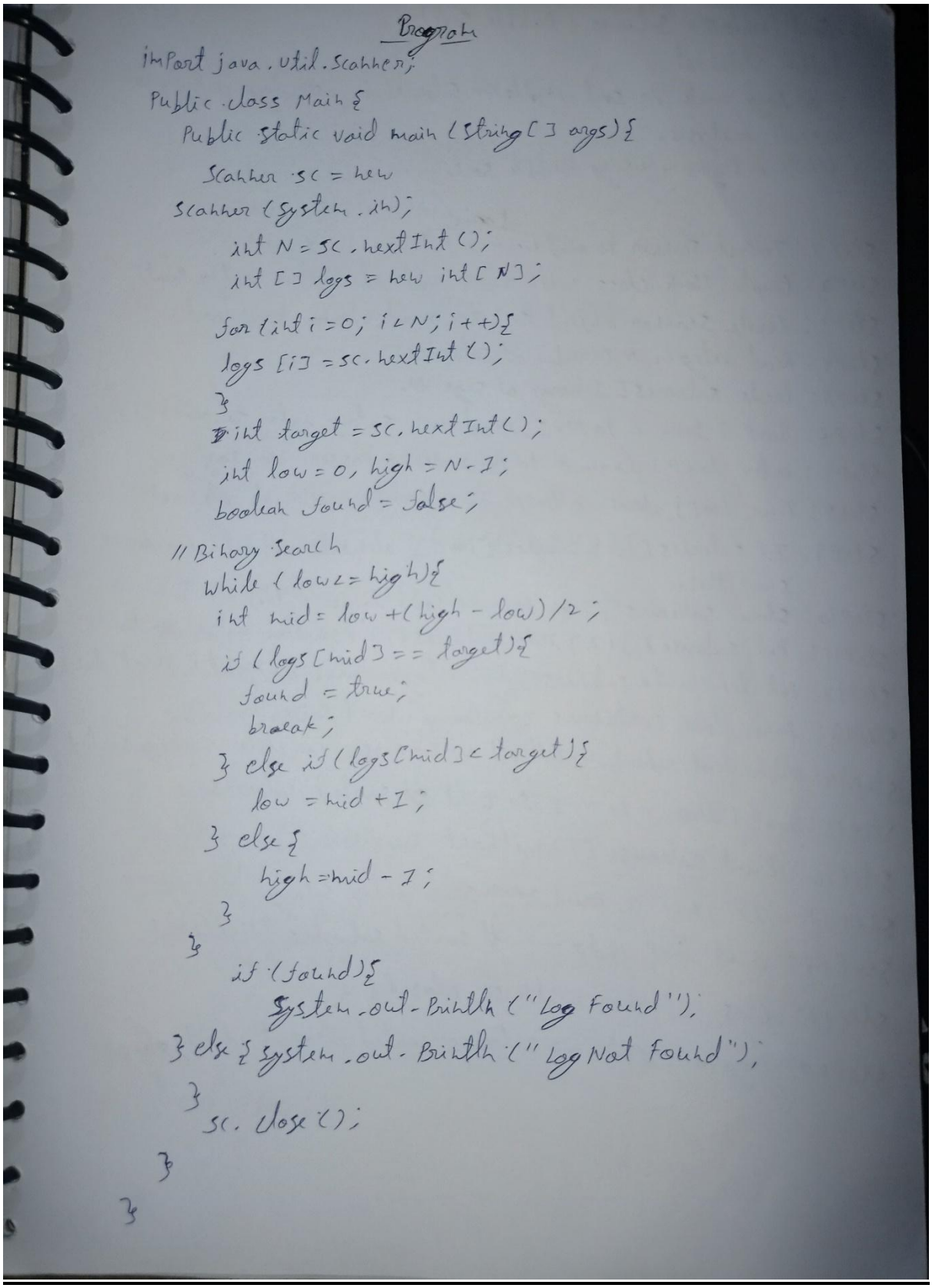


Logic

Step 1:- Import Scanner to read input.
Step 2:- Create Main class with main method
Step 3:- Create Scanner object sc for keyboard input
Step 4:- Read N (number of log IDs)
Step 5:- Create logs[] array of size N,
Step 6:- Loop i from 0 to N-1, read each log ID into logs[i].
Step 7:- Read target (the log ID to search for)
Step 8:- Set low = 0 (start index) and high = N-1 (end index)
Step 9:- Set found = false (assume not found ~~initially~~ initially)
Step 10:- Start Binary search with while loop: continue while low <= high.
Step 11:- Calculate mid = low + (high - low)/2 (middle index).
Step 12:- Check if logs[mid] == target (middle matches target).
Step 13:- If match; set found = true and break (exit loop)
Step 14:- If logs[mid] < target: target is in right half → set low = mid + 1
Step 15:- If logs[mid] > target: target is in left half → set high = mid-1
Step 16:- Loop repeats with new low or high until found or low > high
Step 17:- After loop ends, check found flag.
Step 18:- If found = true, Print "Log found".
Step 19:- If found = false, Print "Log Not Found".
Step 20:- Close scanner with sc.close()
Step 21:- Program ends.

Week 2 Module:

Program



```
Program
import java.util.Scanner;
Public class Main {
  Public static void main (String[] args){
    Scanner sc = new
  Scanner (System.in);
      int N = sc.nextInt();
      int[] logs = new int[N];
    for(int i=0; i<N; i++){
    logs[i] = sc.nextInt();
    }
    int target = sc.nextInt();
    int low = 0, high = N-1;
    boolean found = false;
  // Binary Search
    while (low<=high){
    int mid = low+(high-low)/2;
    if (logs[mid] == target){
      found = true;
      break;
    } else if (logs[mid]<target){
      low = mid+1;
    } else{
      high = mid-1;
    }
  }
    if (found){
      System.out.println("Log Found");
  } else { system.out.println("Log Not Found");
  }
  sc.close();
  }
}
```

Week 2 Module:

Output

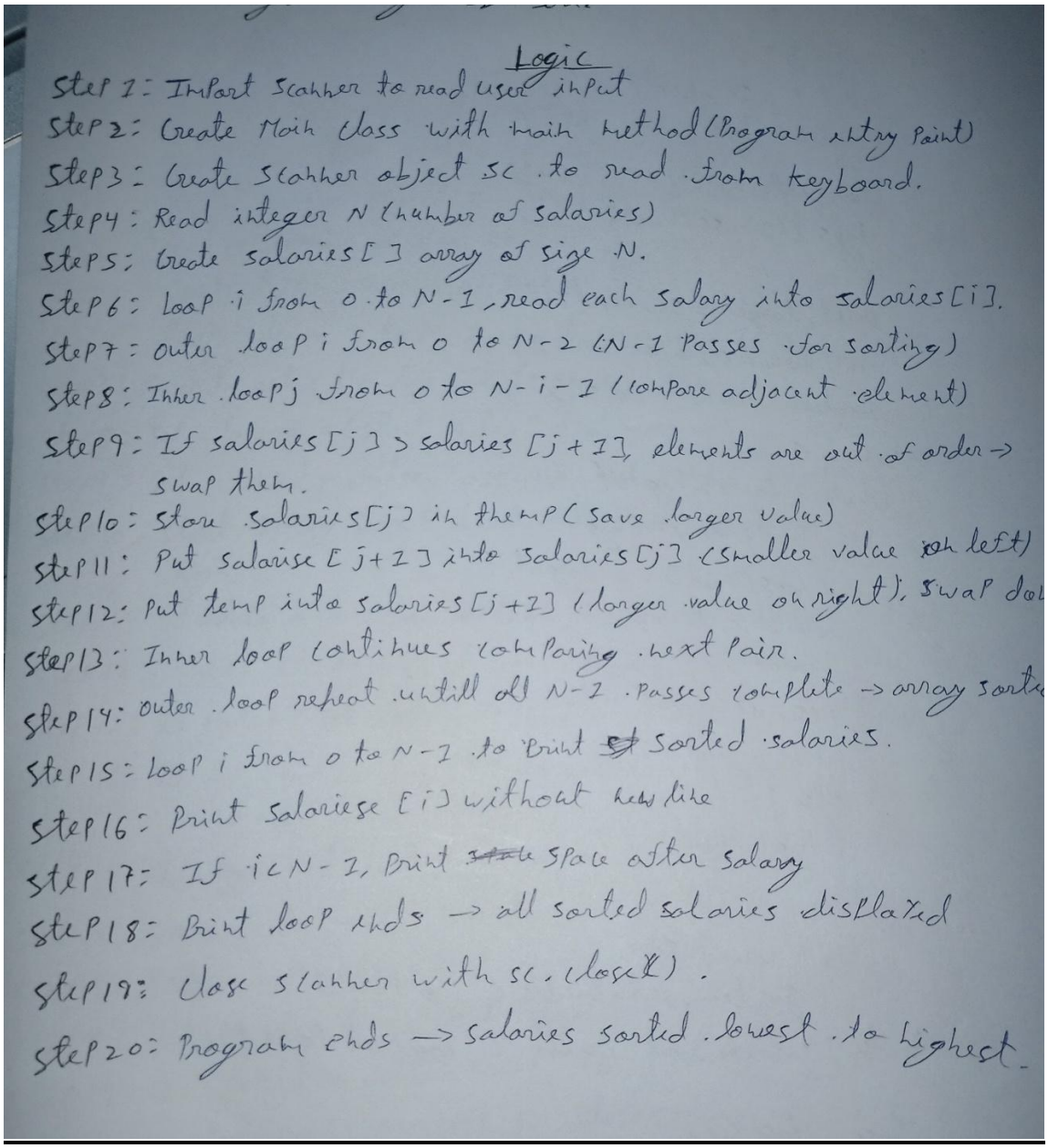
Week 2 Module:

3.Sort Employee Salaries (bubble sort)

Problem Statement

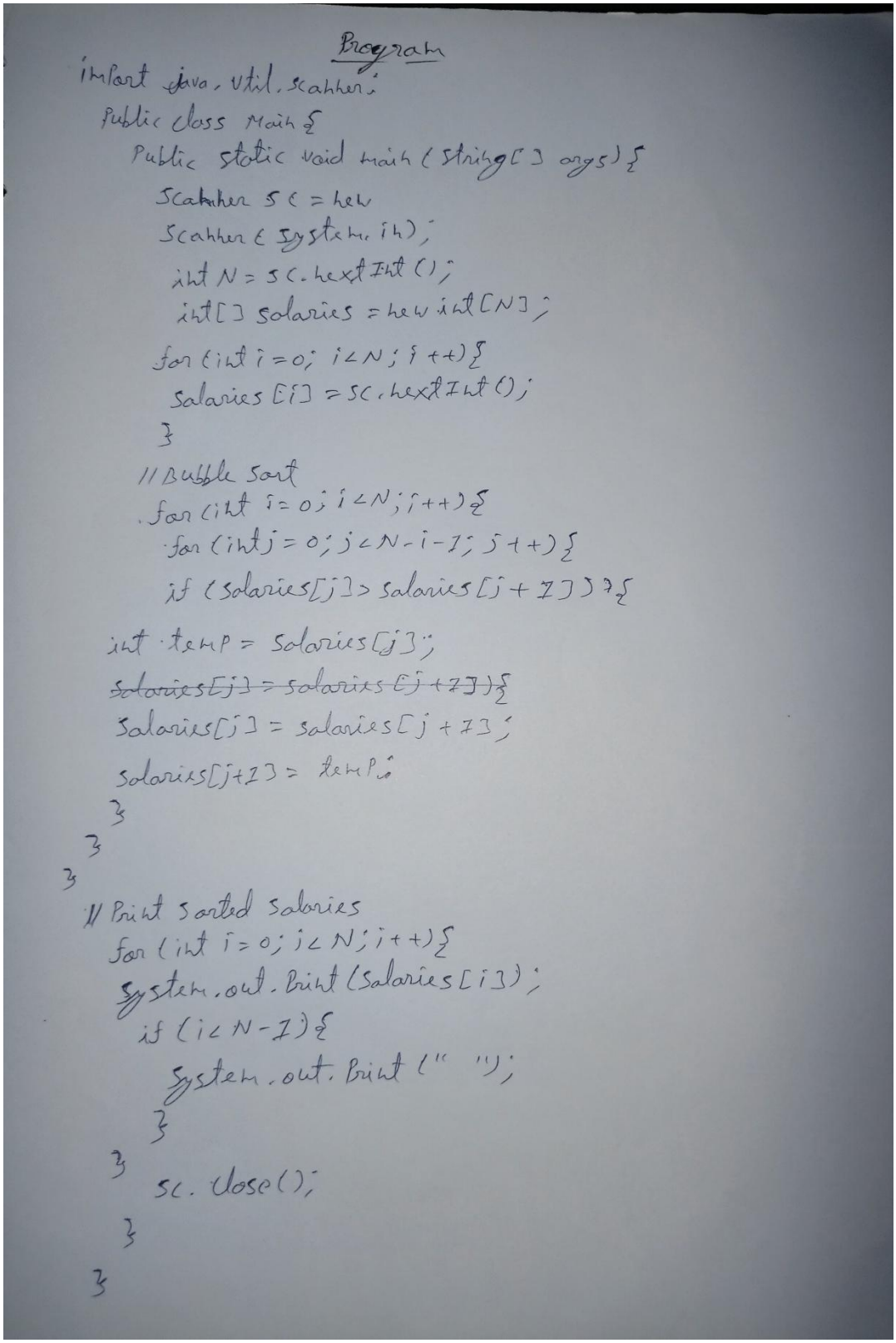
A company wants to sort employee salaries in ascending order for payroll analysis.

Logic



Week 2 Module:

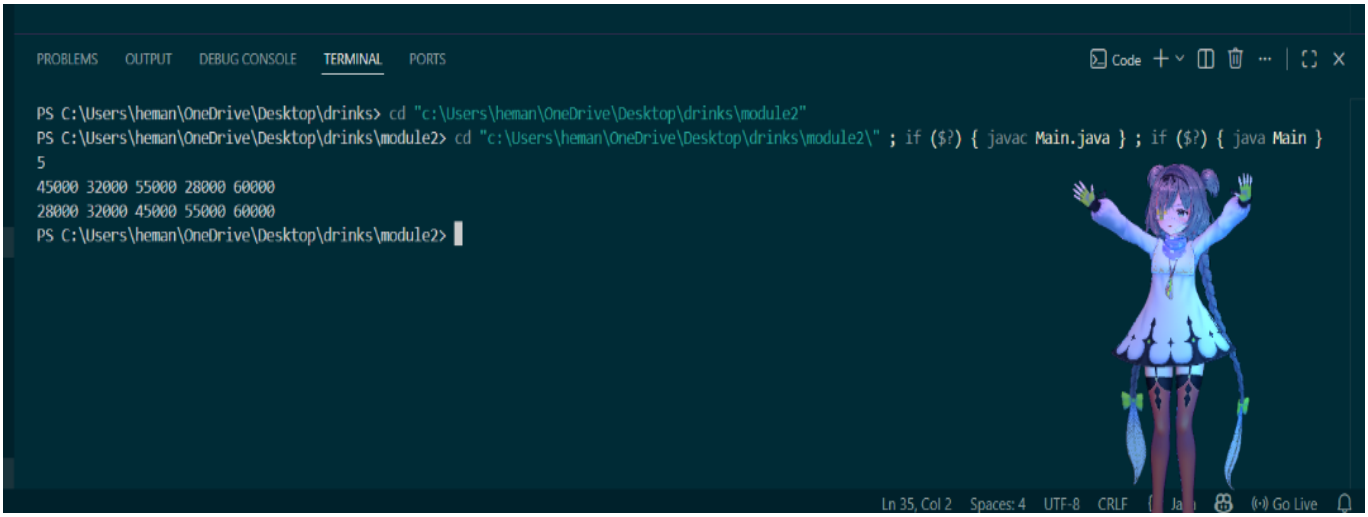
Program



```
Program
import java.util.Scanner;
Public class Main{
  Public static void main(String[] args){
    Scanner sc = new
    Scanner(System.in);
    int N = sc.nextInt();
    int[] salaries = new int[N];
   for(int i=0; i<N; i++){
    salaries[i] = sc.nextInt();
   }
   // Bubble sort
   for(int i=0; i<N; i++){
    for(int j=0; j<N-i-1; j++){
    if(salaries[j] > salaries[j+1]){
int temp = salaries[j];
salaries[j] = salaries[j+1]){
salaries[j] = salaries[j+1];
salaries[j+1] = temp;
  }
}
}
// Print sorted salaries
  for(int i=0; i<N; i++){
  System.out.Print(salaries[i]);
    if(i<N-1){
      System.out.Print(" ");
    }
  }
    sc.close();
  }
}
```

Week 2 Module:

Output



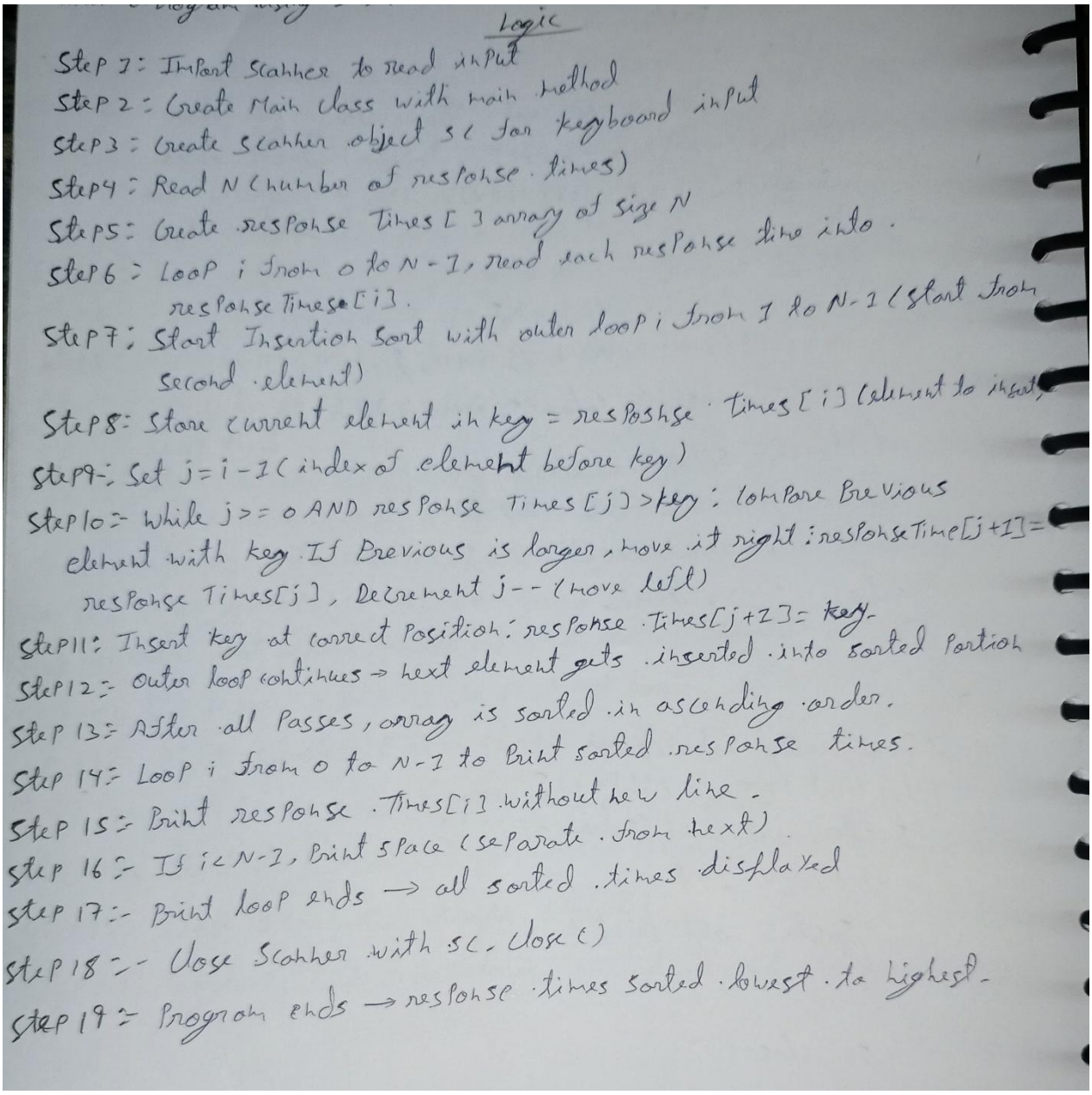
Week 2 Module:

4.Sort Website Response Time (Insertion Sort)

Problem Statement

A web monitoring system records website response times. New data comes continuously and must be inserted into the correct position.

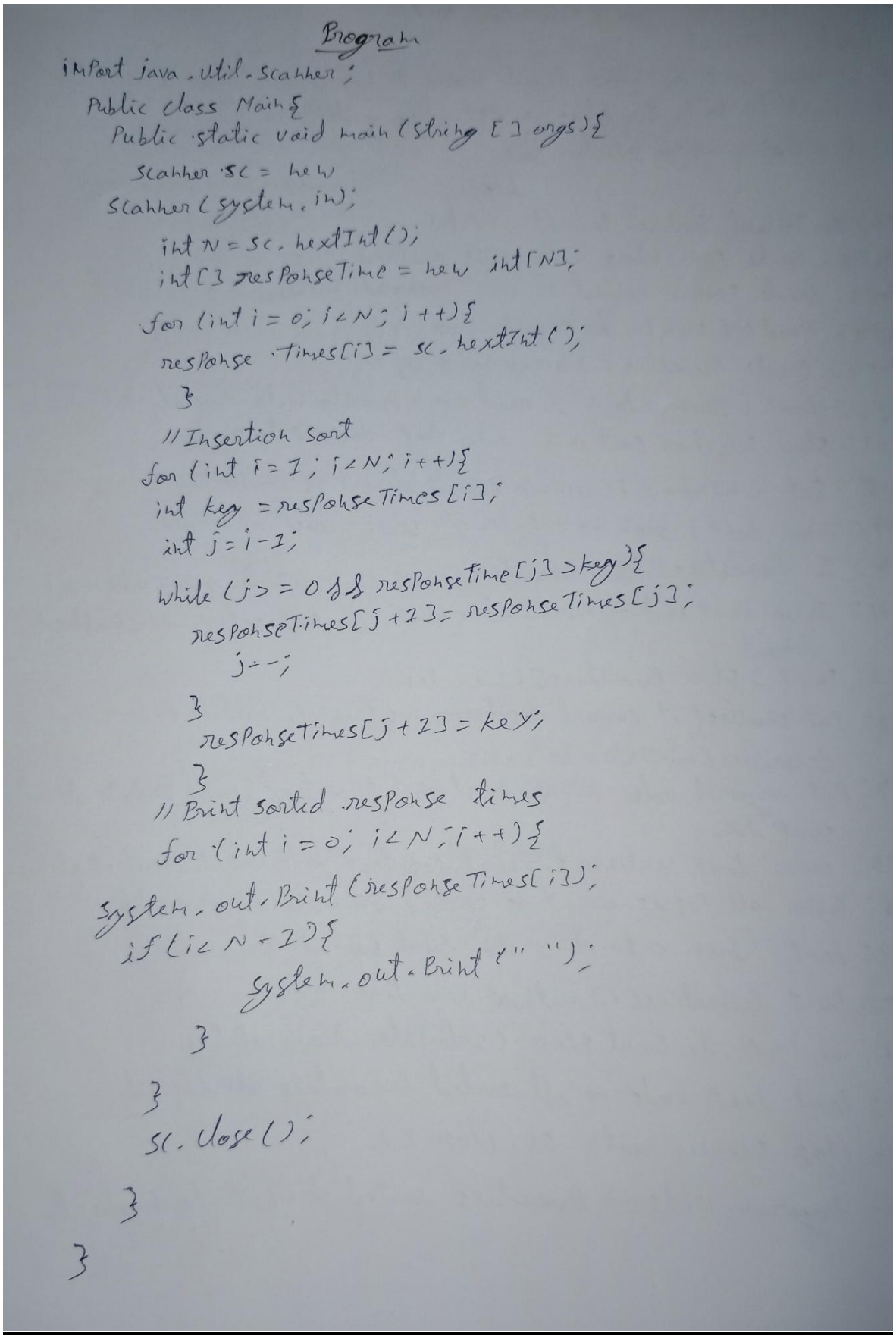
Logic


Logic

Step 1: Import Scanner to read input
Step 2: Create Main class with main method
Step 3: Create Scanner object sc for keyboard input
Step 4: Read N (number of response times)
Step 5: Create responseTimes[] array of size N
Step 6: Loop i from 0 to N-1, read each response time into responseTimes[i].
Step 7: Start Insertion Sort with outer loop i from 1 to N-1 (start from second element)
Step 8: Store current element in key = responseTimes[i] (element to insert)
Step 9: Set j = i-1 (index of element before key)
Step 10: while j >= 0 AND responseTimes[j] > key: (compare previous element with key. If previous is larger, move it right: responseTimes[j+1] = responseTimes[j], Decrement j-- (move left)
Step 11: Insert key at correct position: responseTimes[j+1] = key.
Step 12: Outer loop continues → next element gets inserted into sorted portion
Step 13: After all passes, array is sorted in ascending order.
Step 14: Loop i from 0 to N-1 to print sorted response times.
Step 15: Print responseTimes[i] without new line.
Step 16: If i < N-1, print space (separate from next)
Step 17: Print loop ends → all sorted times displayed
Step 18: Close Scanner with sc.close()
Step 19: Program ends → response times sorted lowest to highest.

Week 2 Module:

Program



```
Program
import java.util.Scanner;
Public class Main{
  Public static void main(String[] args){
    Scanner sc = new
  Scanner(System.in);
      int N = sc.nextInt();
      int[] responseTime = new int[N];
    for(int i=0; i<N; i++){
      response Times[i] = sc.nextInt();
      }
      // Insertion sort
    for(int i=1; i<N; i++){
      int key = responseTimes[i];
      int j=i-1;
      while(j>=0 && responsetime[j]>key){
        responseTimes[j+1]= responseTimes[j];
          j--;
        }
        responsetimes[j+1]=key;
      }
      // Print sorted response times
      for(int i=0; i<N;i++){
System.out.Print(responseTimes[i]);
  if(i<N-1){
          System.out.Print(" ");
      }
    }
    sc.close();
  }
}
```

Week 2 Module:

Output

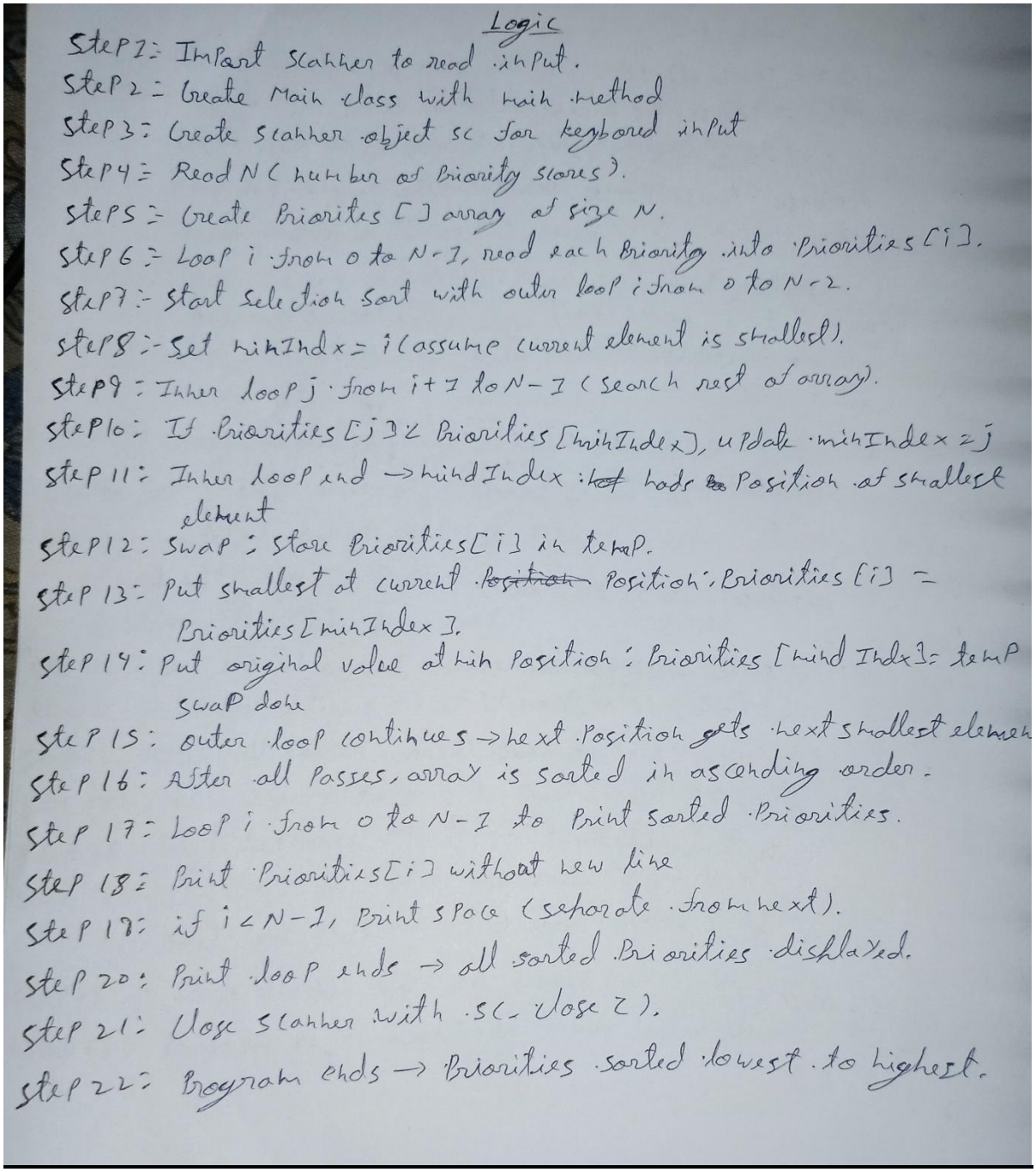
Week 2 Module:

5.Prioritize Bug Reports (Section Sort)

Problem Statement

A software company assign priority scores to bug reports. To resolve bugs efficiently, they want to sort priorities in ascending order.

Logic

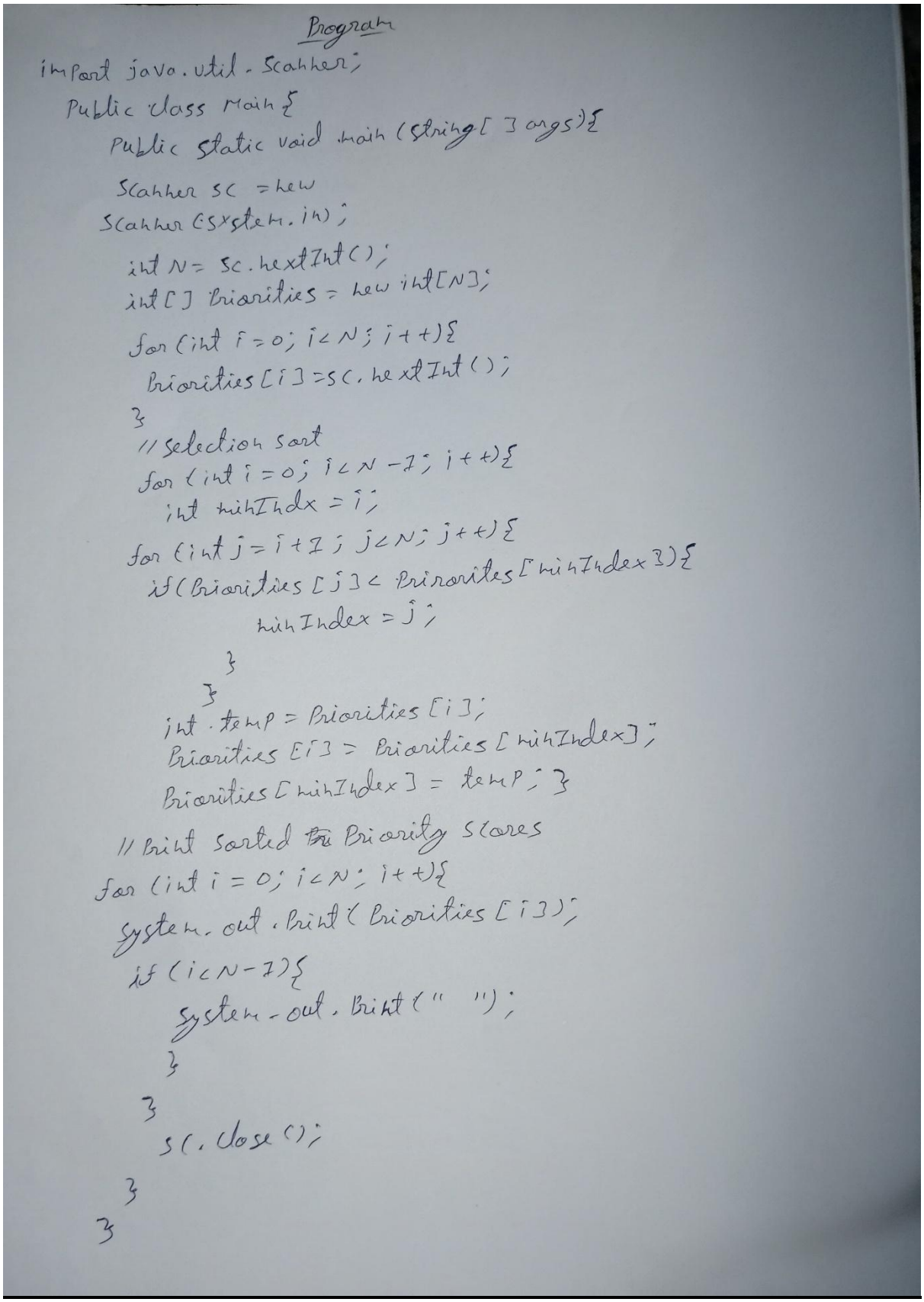


Logic

Step 1: Import Scanner to read input.
Step 2: Create Main class with main method
Step 3: Create Scanner object sc for keyboard input
Step 4: Read N (number of Priority scores).
Step 5: Create Priorites [] array of size N.
Step 6: Loop i from 0 to N-1, read each Priority into Priorities[i].
Step 7: Start Selection Sort with outer loop i from 0 to N-2.
Step 8: Set minIndex = i (assume current element is smallest).
Step 9: Inner loop j from i+1 to N-1 (search rest of array).
Step 10: If Priorities[j] < Priorities[minIndex], update minIndex = j
Step 11: Inner loop end → minIndex: ~~not~~ holds the Position of smallest element
Step 12: Swap : Store Priorities[i] in temp.
Step 13: Put smallest at current ~~Position~~ Position: Priorities[i] = Priorities[minIndex].
Step 14: Put original value at min position : Priorities[minIndex] = temp
Swap done
Step 15: outer loop continues → next position gets next smallest element
Step 16: After all passes, array is sorted in ascending order.
Step 17: Loop i from 0 to N-1 to Print sorted Priorities.
Step 18: Print Priorities[i] without new line
Step 19: if i < N-1, Print space (separate from next).
Step 20: Print loop ends → all sorted Priorities displayed.
Step 21: Close Scanner with sc.close().
Step 22: Program ends → Priorities sorted lowest to highest.

Week 2 Module:

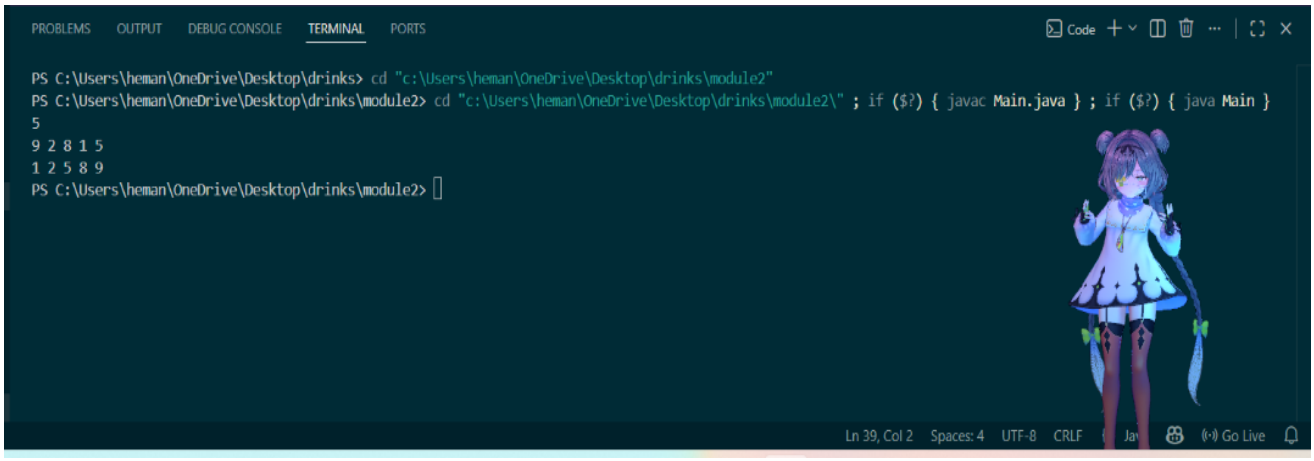
Program



```
Program
import java.util.Scanner;
Public class Main{
    Public static void main(String[] args){
    Scanner sc = new
    Scanner(System.in);
     int N= sc.nextInt();
     int[] Priorities = new int[N];
     for(int i=0; i<N; i++){
      Priorities[i]=sc.nextInt();
     }
     // Selection sort
     for(int i=0; i<N-1; i++){
       int minIndx = i;
     for(int j=i+1; j<N; j++){
      if(Priorities[j]< Priorites[minIndex]){
            minIndex = j;
          }
        }
       int temp = Priorities[i];
       Priorities[i] = Priorities[minIndex];
       Priorities[minIndex] = temp; }
    // Print sorted Priority scores
   for(int i=0; i<N; i++){
    System.out.Print(Priorities[i]);
     if(i<N-1){
        System.out.Print(" ");
        }
     }
       sc.close();
   }
}
```

Week 2 Module:

Output



Week 2 Module:

Week 2 Module: